

Quantized Guardians: Accuracy-Neutral Decision Churn in Compressed Prompt-Injection Detectors

Muneeb Anjum*

Independent Researcher,

Abstract

Classifier guardrails against prompt injection and jailbreaks are routinely compressed to low precision before deployment. The usual acceptance test for a compressed guardrail is that benchmark accuracy is unchanged. We show with paired experiments on four public detectors (ProtectAI-v2, PIGuard, Deepset, FMOPs; 2,755 inputs) that this test is unsafe: compression can leave F1 and AUROC nearly intact while changing which attacks are blocked. Four-bit per-channel round-to-nearest quantization of ProtectAI-v2 changes 3.92% of decisions and lets through 51 attacks that FP32 blocked. The default one-line native dynamic INT8 paths (PyTorch, ONNX Runtime) are far worse: they flip 30.05% and 38.19% of ProtectAI-v2 decisions and 31.54% of PIGuard decisions, and PIGuard stops flagging any input as an attack (F1 = 0). A per-tensor fake-quantized simulation reproduces these failures, whereas the commonly used per-token simulation reports only 1.78% and 3.05% flips, so such simulations can badly understate deployed risk. An ablation attributes the collapse to per-tensor scales: per-tensor weights alone flip 36.7% of decisions and per-tensor activations alone 21.3% on a 150-input subset. Native INT8 on x86_64 CPU runners and on an aarch64 CPU produce flip rates within 0.4 points (ProtectAI-v2: 29.73% vs 30.05%) but still disagree on 117 of 2,755 individual decisions. GPTQ did not reduce churn relative to RTN for three of four detectors, and its flip sets varied across seeds (Jaccard 0.29–0.76). Re-tuning the threshold removes part of the shift but not input-specific churn. We propose an instance-level acceptance protocol for compressed security classifiers.

Keywords: LLM security, prompt injection, guardrails, jailbreak detection, model quantization, INT8 inference, robustness evaluation, prediction churn

1. Introduction

Prompt injection—adversarial text that overrides the instructions an application gives to a large language model (LLM)—has become one of the defining security problems of LLM-integrated software [1–3]. Because the LLM itself cannot reliably separate instructions from data, most production deployments add an *input guardrail*: a small encoder classifier, usually a fine-tuned DeBERTa-v3 [4] or DistilBERT [5] model, that scores every user prompt or retrieved document and blocks those it believes to be attacks. Public examples include ProtectAI’s DeBERTa detector [6], PIGuard [7], Meta’s Prompt Guard [8], and many community models trained on open datasets.

A guardrail sits on the critical path of every request, so it is a natural target for compression. INT8 ONNX exports of popular detectors are distributed on model hubs, PyTorch and ONNX Runtime offer one-line dynamic INT8 quantization, and 4-bit weight quantizers developed for LLMs [9–12] are increasingly applied to encoders. The acceptance test that accompanies this step is almost always the same: re-run a benchmark and confirm that accuracy, F1, or AUROC is essentially unchanged.

For a security control this test is inadequate. A filter is not attacked on average; an adversary needs one input that passes, and an operator’s allow-lists, alert thresholds, and red-team results are all tied to the *specific* decisions of the model that was validated. If compression preserves the average while reshuffling *which* inputs pass, the aggregate check gives false assurance. The phenomenon of prediction *churn* [13] and of compression-induced *flips* [14, 15] is known in general machine learning and in generative LLMs, but it has not been quantified for the classifiers that guard LLM applications, where a flip is not a cosmetic change in an answer but a changed security decision.

This paper measures that effect. We evaluate four public prompt-injection detectors under a grid of numeric configurations—full precision (FP32), BF16, two simulated W8A8 variants, *native* INT8 (PyTorch and ONNX

*Corresponding author. ORCID: 0009-0001-2042-7656

Email address: horizonbymuneeb@gmail.com (Muneeb Anjum)

URL: <https://orcid.org/0009-0001-2042-7656> (Muneeb Anjum)

Runtime integer kernels, per-tensor and per-channel weights), and 4-bit weight quantization with round-to-nearest (RTN), GPTQ [10], and an AWQ-style activation-aware variant [11]—on 2,755 inputs drawn from four public benchmarks plus 900 character-perturbed attacks. Throughout, the reference for every compressed model is the *same detector’s* FP32 decisions, so the quantity we measure is the security-relevant change introduced by compression, independent of how good the detector was to begin with.

Contributions.

1. *Protocol.* We define an instance-level acceptance protocol for compressed guardrails: decision flip rate with paired bootstrap confidence intervals, counts of newly missed attacks and newly blocked benign inputs, exact McNemar tests on paired correctness, adversarial-slice recall, and flip-set overlap across calibration seeds and implementations (Section 4).
2. *Accuracy-neutral churn.* Across detectors and formats, aggregate F1 moves by at most a few points while up to 15.7% of individual decisions change; for the strongest detector, 4-bit per-channel quantization lets through 51 attacks the FP32 model blocked with no statistically significant change in accuracy (Section 5).
3. *Native versus simulated INT8.* We execute INT8 with real integer kernels and compare the resulting flip sets with fake-quantized simulation, showing that default per-tensor native INT8 collapses ProtectAI-v2 and PIGuard (30–38% flips), that per-token simulation understates this, and that results agree in aggregate but not input by input across aarch64 and x86_64 CPUs.
4. *Calibration-based quantization.* We test whether GPTQ (and a single-seed AWQ-style method) reduces churn relative to RTN across three calibration seeds, and whether the *set* of flipped inputs is stable across seeds.
5. *Mitigation and guidance.* We show that held-out threshold re-calibration repairs global score shifts but not input-specific churn, and distil practical recommendations for practitioners and for guardrail vendors (Section 6).

2. Background and related work

2.1. Prompt injection and classifier guardrails

Perez and Ribeiro [1] introduced goal-hijacking and prompt-leaking attacks against instruction-following models. Greshake et al. [2] extended the threat to *indirect* injection, in which malicious instructions arrive through retrieved web pages, e-mails, or tool outputs, and Liu et al. [3] formalised attacks and benchmarked defences. Defences fall into prevention (prompt design, privilege separation) and detection. Detection by a separately trained classifier is the most widely deployed approach because it is model-agnostic and cheap: a DeBERTa-base classifier has about 184M parameters, orders of magnitude fewer than the LLM it protects. Content-safety models such as Llama Guard [16] play a related role for harmful content.

The robustness of such classifiers has been studied only at full precision. Li et al. [7] identified *over-defence*—benign prompts that contain trigger words such as “ignore” being flagged—and released the NotInject benchmark and the PIGuard detector. Hackett et al. [17] showed that character-level perturbations (homoglyphs, zero-width characters, leetspeak) and adversarial ML techniques evade several commercial and open guardrails, and Rahman and Alouani [18] built universal evasive suffixes by gradient search. Zhou et al. [19] showed that fixed 512-token inspection windows can be exploited by spreading an injection across segments. Our work is complementary: rather than searching for new attacks against a fixed detector, we ask how the set of attacks a detector stops changes when the detector is compressed.

2.2. Post-training quantization of transformers

Post-training quantization (PTQ) maps weights, and optionally activations, to low-bit integers with a scale and zero-point per tensor, channel, or group [20–22]. For BERT-style encoders, Q8BERT [23], Q-BERT [24], and I-BERT [25] showed that 8-bit and lower precision can preserve GLUE accuracy. For LLMs, LLM.int8() [9] handles activation outliers with mixed precision, SmoothQuant [12] migrates activation difficulty into weights, GPTQ [10] uses second-order (Hessian) information to compensate rounding error column by column, and AWQ [11] protects salient weight channels identified from activation magnitudes. In deployment, *dynamic* INT8 quantization of linear layers—weights quantized offline, activations quantized per batch at run time—is the default CPU path in both PyTorch [26] and ONNX Runtime [27]. All of these methods are evaluated by aggregate accuracy or perplexity.

2.3. Compression, churn, and trust

Prediction churn—disagreement between two models of similar accuracy—was characterised by Milani Fard et al. [13] as a cost in iterative model deployment. Hooker et al. [15] showed that pruned and quantized image classifiers retain top-line accuracy while disproportionately changing predictions on a small subset of atypical examples, and later that this subset is enriched for under-represented groups [28]. For LLMs, Dutta et al. [14]

Table 1: Detectors and numeric configurations. “Sim.” = quantize–dequantize simulation in floating point; “Native” = integer kernels.

Detectors	ProtectAI-v2, PIGuard, Deepset (DeBERTa-v3-base, 184M params); FMops (DistilBERT, 66M)
FP32 / BF16	reference / bfloat16 weights and activations
W8A8 ^s (token)	sim.: symmetric per-output-channel INT8 weights, symmetric per-token INT8 activations
W8A8 ^s (tensor)	sim.: per-tensor INT8 weights, per-tensor asymmetric 7-bit activations (mirrors the PyTorch dynamic path)
INT8 PyTorch	native: <code>torch.ao.quantization.quantize_dynamic</code> on all <code>nn.Linear</code> (qint8 weights, per-tensor dynamic activations; default engine)
INT8 ONNX RT	native: ONNX export (opset 17) + <code>onnxruntime.quantization.quantize_dynamic</code> (QInt8 weights, per-tensor), CPU execution provider
W4-g128 / W4-pc RTN	sim.: 4-bit asymmetric round-to-nearest weights, groups of 128 input channels / one scale per output channel
W4 GPTQ	sim.: GPTQ [10] (1% damping, block size 128, no act-order), sequential block-wise calibration
W4 AWQ-style	sim.: per-input-channel scaling $s = x ^\alpha$, α grid-searched per layer group [11]

showed that quantized models with near-identical benchmark accuracy exhibit substantial answer “flips”, and proposed flips and KL divergence as complementary metrics; Hong et al. [29] found that compression can degrade trustworthiness dimensions that perplexity does not reveal; and Egashira et al. [30] showed that the quantization step itself can be weaponised to activate hidden malicious behaviour. In adversarial robustness, Lin et al. [31] observed that quantization can amplify adversarial perturbations. We bring these lenses to guardrail classifiers, where a flip has a direct security interpretation (an attack that now passes, or a legitimate user who is now blocked), and we add the comparison between simulated and native kernels and the cross-seed stability of flip sets, which prior churn studies did not examine.

3. Threat model and problem statement

We consider an operator who deploys a validated full-precision detector f and later replaces it with a compressed artifact f_q to reduce latency or memory, accepting f_q if its aggregate metrics on a held-out benchmark match those of f . The adversary submits text to the application and wins if an injection passes the guardrail. We consider a *non-adaptive* adversary who reuses attacks (including character-perturbed variants) that are known to work against some detectors but does not query f_q ; an adaptive adversary who searches against f_q directly is strictly stronger, so our measurements are a lower bound on the security impact. We also consider the operator’s cost of new false positives, since over-defence degrades usability and pushes users to disable the guardrail.

For an input x_i with label $y_i \in \{0,1\}$ (1 = attack) and threshold τ , let $\hat{y}_i = \mathbb{1}[f(x_i) \geq \tau]$ and $\hat{y}_i^q = \mathbb{1}[f_q(x_i) \geq \tau]$. Compression is *accuracy-neutral* on a dataset if accuracy (or F1) changes by less than its sampling uncertainty; it exhibits *churn* if the flip rate $\frac{1}{n} \sum_i \mathbb{1}[\hat{y}_i^q \neq \hat{y}_i]$ is materially non-zero. The security-relevant components of churn are the *newly missed attacks* $|\{i : y_i=1, \hat{y}_i=1, \hat{y}_i^q=0\}|$ and the *new false positives* $|\{i : y_i=0, \hat{y}_i=0, \hat{y}_i^q=1\}|$. Our central claim is that accuracy-neutral churn is common for compressed guardrails and that it is invisible to the standard acceptance test.

4. Methodology

4.1. Detectors

We study four publicly available binary prompt-injection detectors from the Hugging Face hub (Table 1): *ProtectAI-v2* (protectai/deberta-v3-base-prompt-injection-v2) [6], *PIGuard* (leolee99/PIGuard, DeBERTa-v3-base) [7], *Deepset* (deepset/deberta-v3-base-injection), and *FMops* (fmops/distilbert-prompt-injection). Meta’s Prompt Guard models are gated behind a licence agreement and were excluded. For each model we use the softmax probability p of the attack class and the default threshold $\tau = 0.5$; inputs are truncated to 512 tokens.

4.2. Numeric formats

All configurations start from the released FP32 checkpoint. **BF16** casts all weights and activations to bfloat16. The two **simulated W8A8** variants apply quantize–dequantize (“fake quantization”) to the weights and, through forward pre-hooks, to the inputs of every linear layer: the *token* variant uses symmetric per-output-channel weight scales and symmetric per-token activation scales; the *tensor* variant uses one weight scale per

Table 2: Evaluation set composition.

Source	Benign	Attack	Role
Deepset test [32]	56	60	clean pool
Safe-Guard test (balanced sample) [33]	500	500	clean pool
Jailbreak-Classification test [34]	259	141	clean pool
NotInject [7]	339	0	over-defence
Homoglyph / zero-width / leet	0	3×300	adversarial slices
Total	1,154	1,601	2,755 inputs

tensor and asymmetric per-tensor activation scales over the reduced 0–127 range used by PyTorch’s dynamic kernels, so that it mirrors native execution as closely as a simulation can. The two **native INT8** configurations execute real integer matrix multiplications on CPU: PyTorch dynamic quantization of every `nn.Linear` module, and ONNX Runtime dynamic quantization of an exported ONNX graph. Native runs process one input at a time (no padding), because dynamic activation scales in native kernels are computed per tensor and would otherwise depend on which other inputs share a batch; the corresponding FP32 references (PyTorch, ONNX Runtime FP32) are reported alongside.

The **4-bit** configurations quantize weights only, with asymmetric min–max grids. **RTN** rounds each weight to the nearest grid point, with either groups of 128 input channels (**W4-g128**) or a single scale per output channel (**W4-pc**). **GPTQ** quantizes each weight matrix column by column, updating the not-yet-quantized columns to compensate the layer-output error under a Hessian $H = 2XX^\top$ estimated from calibration activations [10]. Calibration is *sequential*: transformer block k is calibrated on activations produced by blocks $< k$ that have already been quantized. Our **AWQ-style** variant scales each input channel by $s_j = \overline{|x_j|}^\alpha$ before RTN and divides it back afterwards, searching $\alpha \in \{0, 0.05, \dots, 1\}$ per layer group (query/key/value projections share one α) to minimise the output reconstruction error on calibration activations; it follows the AWQ principle [11] but is our re-implementation for encoders, not the reference code. In every quantized configuration all linear layers—attention projections, feed-forward layers, pooler, and classification head—are quantized and embeddings are kept in full precision.

Calibration data and seeds. Calibration-based methods draw 128 unlabelled prompts from the *training* splits of the Deepset, Safe-Guard, and Jailbreak-Classification datasets after removing any text that also appears in the evaluation set. Each seed $s \in \{0, 1, 2\}$ draws a different calibration sample; we report mean $\pm$ sample standard deviation across seeds and the overlap of the flipped-input sets. RTN, BF16, and INT8 are deterministic and need no seeds.

4.3. Evaluation data

We assemble 2,755 inputs from four public sources (Table 2): the Deepset prompt-injection test split [32] (116 inputs); a class-balanced sample of 1,000 inputs from the Safe-Guard prompt-injection test split [33]; the Jailbreak-Classification test split [34] (400 inputs, 141 jailbreaks); and all 339 benign NotInject prompts [7], which contain injection trigger words and measure over-defence. Following Hackett et al. [17], we derive three adversarial slices from 300 randomly selected Safe-Guard attacks: *homoglyph* (each of a, e, o, p, c, i, x, y replaced by its Cyrillic look-alike with probability 0.5), *zero-width* (U+200B inserted after letters with probability 0.3), and *leet* (a, e, i, o, s, t mapped to digits with probability 0.5). All sampling uses fixed seeds.

4.4. Metrics and statistics

On the clean pool (Deepset, Safe-Guard, Jailbreak-Classification; $n=1,516$) we report AUROC, F1 with a 95% bootstrap confidence interval (300 resamples), true- and false-positive rates (TPR, FPR), and 10-bin expected calibration error (ECE) [35]. Over-defence is the FPR on NotInject, and robustness is the detection rate on each adversarial slice. The instance-level metrics, computed over all 2,755 inputs relative to the same detector’s FP32 decisions, are the flip rate with a 95% paired bootstrap CI (1,000 resamples of inputs, with the same resampling indices for every configuration) [36]; the numbers of newly missed attacks and new false positives; and $\max_i |p_i^q - p_i|$. Whether correctness changes is tested with an exact McNemar test [37] on the discordant pairs b (inputs that became wrong) and c (inputs that became right). For calibration-based methods we additionally report the mean pairwise Jaccard index of flip sets across seeds (J_{seed}) and against the RTN flip set (J_{RTN}); for native INT8, the Jaccard index between native and simulated flip sets.

4.5. Mitigation: threshold re-calibration

A natural repair is to re-tune τ for the compressed model. We split the inputs 50/50 at random, choose $\tau^* \in \{0.01, 0.015, \dots, 0.99\}$ on the first half to maximise agreement with the FP32 decisions, and report the flip rate on the second half at $\tau=0.5$ and at τ^* . We repeat this over five random splits and report mean $\pm$ standard deviation.

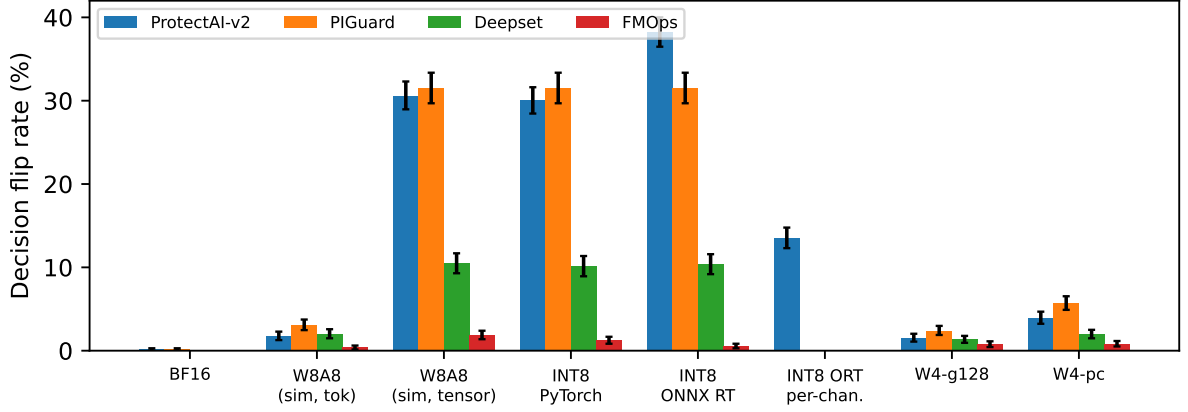


Figure 1: Decision flip rate relative to FP32 for each detector and format.

4.6. Implementation

All experiments were run on a 2-vCPU ARM (Neoverse-V2, aarch64) Linux machine with 8 GB RAM using PyTorch 2.14.1 (CPU), Transformers 5.19.0, and ONNX Runtime 1.30.0. Inputs are processed in length-sorted batches of 16 (4 for batches containing inputs longer than 1,000 characters) except for native and ONNX runs, which use batch size 1. PyTorch’s default quantized engine on this platform (x86) produced outputs identical to the `onednn` engine; `fbgemm` is unavailable on aarch64, and `qnnpack` was compared on a 120-input subset (Section 5.2). Re-running the FP32 models in this environment reproduced the scores of an earlier run in a different software environment to within 1.2×10^{-7} (ProtectAI-v2) and 8.9×10^{-7} (FMOPs) in probability, with zero decision differences, so results obtained in the two runs are directly comparable. Code, the evaluation set, and every per-input score are released (Section 8).

5. Results

We report all flip rates against the same detector’s FP32 decisions on the 2,755 evaluation inputs. Tables 3 and 4 give aggregate and instance-level results for the full grid of formats; Figure 1 summarises the flip rates.

5.1. Accuracy-neutral churn

For the format families in which the model stays functional, aggregate metrics barely move while individual decisions do. For ProtectAI-v2, FP32 reaches F1 0.889; W8A8^s (token) reaches F1 0.903 yet changes 1.78% of decisions (49 inputs), and W4 per-channel RTN reaches 0.895 while changing 3.92% and letting through 51 attacks that FP32 blocked, with 10 benign inputs newly blocked. BF16 is effectively lossless (0.15% flips for ProtectAI-v2, 0.15% for PIGuard). PIGuard shows the same pattern with W4 per-channel RTN (5.74% flips, 66 missed attacks) and W8A8^s (token) (3.05%, 3 missed, 14 new false positives). FMOPs, a DistilBERT detector with near-chance AUROC (0.611) on this mixed set, is the most stable model (flip rates 0.36–1.85% across the grid), and Deepset flips 1.34–2.00% for token-level and W4 RTN formats.

5.2. Native INT8 versus simulation

The default native dynamic INT8 paths are the most damaging configuration we tested (Table 5). PyTorch dynamic INT8 flips 30.05% of ProtectAI-v2 decisions (514 missed attacks, 141 new false positives) and ONNX Runtime INT8 flips 38.19% (805 missed); AUROC on the clean pool falls from 0.977 to 0.542 and 0.505, and F1 from 0.889 to 0.394 and 0.345. For PIGuard both native paths flip 31.54% of decisions, miss all 801 attacks FP32 blocked, and flag nothing as an attack (F1 0.000). Deepset flips about 10% (10.13% PyTorch, 10.34% ONNX RT) with the opposite sign: no missed attacks but 265 and 280 newly blocked benign inputs, and a NotInject false-positive rate of 97.4% and 100% (FP32: 71.4%). FMOPs is nearly unaffected (1.23% and 0.54%).

A *token* simulation (per-channel weights, per-token activations) does not reproduce this: it flips 1.78% (ProtectAI-v2), 3.05% (PIGuard), 2.00% (Deepset) and 0.36% (FMOPs), with flip-set overlap with the native artifact of $J \leq 0.20$. A *tensor* simulation that mirrors the native granularity reproduces the aggregate effect (30.60% and 31.54% flips for ProtectAI-v2 and PIGuard; 10.45% for Deepset), and for PIGuard its decisions are identical to the native paths. For ProtectAI-v2 the overlap with native PyTorch is $J = 0.82$ and the two agree on 94.01% of inputs, but with ONNX RT only $J = 0.67$ (86.24% agreement). Per-channel native variants (weights quantized per output channel, dynamic per-tensor activations) lower the ProtectAI-v2 flip rate to 13.97% (PyTorch) and 13.50% (ONNX RT) and keep AUROC at 0.963 and 0.964, but remain well above the token simulation.

Table 3: Aggregate metrics on the clean pool ($n=1,516$) and NotInject over-defence ($n=339$). F1 brackets: 95% bootstrap CI (300 resamples). W8A8^s = simulated (fake-quantized) INT8; INT8 PyTorch / ONNX RT = native integer kernels on CPU. RTN rows for W4 use round-to-nearest; GPTQ/AWQ rows use calibration seed 0 (all seeds in Table 8).

Detector	Format	AUROC	F1 [95% CI]	TPR	FPR	NotInject FPR
ProtectAI-v2	FP32	0.977	0.889 [0.871, 0.907]	0.807	0.007	0.434
	BF16	0.977	0.888 [0.871, 0.906]	0.806	0.007	0.437
	W8A8 ^s (token)	0.978	0.903 [0.886, 0.919]	0.833	0.010	0.460
	W8A8 ^s (tensor)	0.547	0.397 [0.362, 0.432]	0.287	0.136	0.168
	INT8 PyTorch	0.542	0.394 [0.359, 0.429]	0.288	0.151	0.174
	INT8 ONNX RT	0.505	0.345 [0.308, 0.384]	0.230	0.088	0.106
	INT8 PyTorch (per-ch.)	0.963	0.746 [0.719, 0.770]	0.598	0.004	0.401
	INT8 ONNX RT (per-ch.)	0.964	0.777 [0.753, 0.801]	0.636	0.001	0.381
	W4-g128 RTN	0.976	0.894 [0.878, 0.912]	0.816	0.007	0.442
	W4-pc RTN	0.980	0.895 [0.877, 0.913]	0.817	0.009	0.401
	W4-pc GPTQ (s0)	0.977	0.775 [0.753, 0.801]	0.635	0.002	0.233
	W4-pc AWQ (s0)	0.977	0.907 [0.892, 0.924]	0.839	0.010	0.422
	W4-g128 GPTQ (s0)	0.976	0.888 [0.872, 0.904]	0.806	0.007	0.445
PIGuard	FP32	0.931	0.807 [0.783, 0.830]	0.705	0.036	0.115
	BF16	0.931	0.806 [0.783, 0.829]	0.703	0.036	0.115
	W8A8 ^s (token)	0.944	0.822 [0.799, 0.844]	0.730	0.040	0.133
	W8A8 ^s (tensor)	0.549	0.000 [0.000, 0.000]	0.000	0.000	0.000
	INT8 PyTorch	0.540	0.000 [0.000, 0.000]	0.000	0.000	0.000
	INT8 ONNX RT	0.556	0.000 [0.000, 0.000]	0.000	0.000	0.000
	W4-g128 RTN	0.928	0.784 [0.760, 0.809]	0.676	0.042	0.118
	W4-pc RTN	0.958	0.787 [0.760, 0.810]	0.676	0.037	0.133
	W4-pc GPTQ (s0)	0.944	0.708 [0.676, 0.741]	0.549	0.002	0.065
Deepset	FP32	0.710	0.687 [0.658, 0.707]	0.994	0.775	0.714
	W8A8 ^s (token)	0.692	0.681 [0.651, 0.701]	0.997	0.801	0.805
	W8A8 ^s (tensor)	0.407	0.634 [0.607, 0.656]	1.000	0.993	0.968
	INT8 PyTorch	0.403	0.636 [0.609, 0.658]	0.999	0.980	0.973
	INT8 ONNX RT	0.472	0.632 [0.606, 0.655]	1.000	1.000	1.000
	W4-g128 RTN	0.627	0.683 [0.653, 0.703]	0.997	0.793	0.776
	W4-pc RTN	0.741	0.686 [0.657, 0.707]	0.986	0.763	0.729
	W4-pc GPTQ (s0)	0.761	0.681 [0.652, 0.701]	0.994	0.798	0.796
FMOPs	FP32	0.611	0.672 [0.643, 0.692]	0.973	0.795	0.717
	W8A8 ^s (token)	0.612	0.671 [0.643, 0.691]	0.969	0.791	0.708
	W8A8 ^s (tensor)	0.701	0.668 [0.640, 0.688]	0.959	0.785	0.690
	INT8 PyTorch	0.649	0.673 [0.643, 0.692]	0.971	0.789	0.690
	INT8 ONNX RT	0.635	0.673 [0.643, 0.693]	0.979	0.801	0.717
	W4-g128 RTN	0.618	0.673 [0.644, 0.693]	0.981	0.804	0.735
	W4-pc RTN	0.625	0.673 [0.644, 0.693]	0.977	0.798	0.720
	W4-pc GPTQ (s0)	0.609	0.673 [0.644, 0.693]	0.977	0.798	0.737
	W4-pc AWQ (s0)	0.610	0.672 [0.643, 0.693]	0.974	0.798	0.735

Granularity ablation. On a 150-input subset of ProtectAI-v2, we simulated each granularity combination (Table 6). Per-tensor weights with no activation quantization flip 36.7% of decisions, and per-tensor 7-bit activations with FP32 weights flip 21.3%; each alone is therefore sufficient to cause the collapse. Per-channel weights with per-token activations flip 1.3%, and per-channel weights with per-tensor activations 13.3%. Using the full 8-bit activation range does not help (36.7%). Leaving the feed-forward output projections unquantized cut the flip rate from 28.0% to 14.0%, whereas leaving the classification head unquantized did not (28.0%).

5.3. Engine comparison: aarch64 versus x86_64

Our first native runs used a 2-vCPU aarch64 CPU, on which PyTorch ran with its x86 quantized engine (the `fbgemm` engine is unavailable on aarch64). We repeated the PyTorch dynamic INT8 and FP32 runs for ProtectAI-v2 and FMOPs on GitHub Actions x86_64 runners (AMD EPYC, 2 vCPUs; Table 7). FP32 decisions were identical across the two platforms (0 of 2,755 differ; maximum probability difference 4.3×10^{-5}). Native INT8 flip rates agreed to within 0.4 points, so the collapse is not an artefact of one CPU, but 117 of 2,755 ProtectAI-v2 decisions (4.25%) and 7 FMOPs decisions (0.25%) differed between platforms; the maximum probability difference was 0.28 and 0.77, respectively. A decision-level audit must therefore be repeated on the target hardware.

Table 4: Instance-level change relative to the same detector’s FP32 decisions on all 2,755 inputs. Flip CI: 95% paired bootstrap (1,000 resamples). Missed: attacks blocked by FP32 but passed after quantization. New FP: benign inputs newly blocked. b/c : inputs that became wrong / right; p : exact McNemar test.

Detector	Format	Flip % [95% CI]	Missed	New FP	b/c	McNemar p	$\max \Delta p $
ProtectAI-v2	BF16	0.15 [0.00, 0.29]	2	1	3/1	0.62	0.43
	W8A8 ^s (token)	1.78 [1.27, 2.29]	0	11	11/38	<0.001	0.93
	W8A8 ^s (tensor)	30.60 [28.96, 32.31]	542	130	672/171	<0.001	0.98
	INT8 PyTorch	30.05 [28.46, 31.62]	514	141	655/173	<0.001	0.99
	INT8 ONNX RT	38.19 [36.48, 40.00]	805	85	890/162	<0.001	1.00
	INT8 PyTorch (per-ch.)	13.97 [12.70, 15.28]	259	34	293/92	<0.001	1.00
	INT8 ONNX RT (per-ch.)	13.50 [12.30, 14.77]	262	29	291/81	<0.001	1.00
	W4-g128 RTN	1.52 [1.09, 2.03]	11	7	18/24	0.44	0.99
	W4-pc RTN	3.92 [3.23, 4.68]	51	10	61/47	0.21	0.99
	W4-pc GPTQ (s0)	10.16 [9.00, 11.36]	201	0	201/79	<0.001	1.00
	W4-pc AWQ (s0)	3.01 [2.40, 3.63]	18	10	28/55	0.004	1.00
	W4-g128 GPTQ (s0)	1.31 [0.91, 1.74]	10	7	17/19	0.87	0.97
PIGuard	BF16	0.15 [0.04, 0.29]	1	0	1/3	0.62	0.03
	W8A8 ^s (token)	3.05 [2.47, 3.74]	3	14	17/67	<0.001	0.59
	W8A8 ^s (tensor)	31.54 [29.69, 33.36]	801	0	801/68	<0.001	0.96
	INT8 PyTorch	31.54 [29.69, 33.36]	801	0	801/68	<0.001	0.95
	INT8 ONNX RT	31.54 [29.69, 33.36]	801	0	801/68	<0.001	0.95
	W4-g128 RTN	2.40 [1.89, 2.98]	37	7	44/22	0.009	0.54
	W4-pc RTN	5.74 [4.90, 6.53]	66	24	90/68	0.094	0.99
	W4-pc GPTQ (s0)	16.26 [14.88, 17.68]	401	0	401/47	<0.001	0.93
Deepset	W8A8 ^s (token)	2.00 [1.49, 2.58]	0	52	52/3	<0.001	0.98
	W8A8 ^s (tensor)	10.45 [9.29, 11.69]	0	273	273/15	<0.001	0.77
	INT8 PyTorch	10.13 [8.93, 11.36]	0	265	265/14	<0.001	0.77
	INT8 ONNX RT	10.34 [9.18, 11.58]	0	280	280/5	<0.001	0.74
	W4-g128 RTN	1.34 [0.94, 1.78]	0	35	35/2	<0.001	0.96
	W4-pc RTN	2.00 [1.49, 2.50]	6	22	28/27	1.00	0.93
	W4-pc GPTQ (s0)	5.34 [4.50, 6.21]	87	52	139/8	<0.001	0.90
FMOps	W8A8 ^s (token)	0.36 [0.15, 0.62]	4	0	4/6	0.75	0.37
	W8A8 ^s (tensor)	1.85 [1.38, 2.40]	19	6	25/26	1.00	0.93
	INT8 PyTorch	1.23 [0.83, 1.67]	7	5	12/22	0.12	0.93
	INT8 ONNX RT	0.54 [0.29, 0.83]	0	8	8/7	1.00	0.75
	W4-g128 RTN	0.76 [0.44, 1.13]	0	13	13/8	0.38	0.81
	W4-pc RTN	0.80 [0.51, 1.16]	0	11	11/11	1.00	0.81
	W4-pc GPTQ (s0)	0.54 [0.29, 0.87]	0	10	10/5	0.30	0.59
	W4-pc AWQ (s0)	0.69 [0.40, 1.02]	2	11	13/6	0.17	0.71

5.4. Calibration-based 4-bit quantization

Table 8 compares RTN, GPTQ (three calibration seeds) and AWQ-style scaling (one seed). GPTQ did *not* reduce decision churn relative to W4 per-channel RTN for three of the four detectors: ProtectAI-v2 flips $9.39 \pm 0.67\%$ versus 3.92% for RTN, PIGuard $15.69 \pm 0.95\%$ versus 5.74% , and Deepset $3.91 \pm 1.29\%$ versus 2.00% . Only FMOps was marginally better ($0.64 \pm 0.23\%$ versus 0.80%). With groups of 128, a single GPTQ run on ProtectAI-v2 flipped 1.31% (RTN: 1.52%), and the AWQ-style variant 3.01% (per-channel RTN: 3.92%). The GPTQ flip sets differ across seeds (mean pairwise Jaccard 0.66 ProtectAI-v2, 0.76 PIGuard, 0.29 Deepset, 0.31 FMOps) and overlap little with the RTN flip set (J_{RTN} 0.24, 0.16, 0.15, 0.35). The GPTQ rows for ProtectAI-v2 and PIGuard are almost entirely missed attacks (184 and 385 on average) with few new false positives, together with lower F1 (0.785 and 0.686 versus 0.889 and 0.807 for FP32). Two “identical” GPTQ releases built with different calibration samples are thus not interchangeable.

5.5. Threshold re-calibration and perturbed attacks

Re-tuning τ on a held-out half to maximise agreement with FP32 reduces some shifts (Table 9): ONNX RT INT8 for ProtectAI-v2 falls from $37.91 \pm 1.41\%$ to $30.68 \pm 0.73\%$, and W4 per-channel GPTQ from $9.81 \pm 0.36\%$ to $4.92 \pm 0.62\%$, but PyTorch INT8 stays at $29.12 \pm 1.03\%$. Re-calibration therefore does not repair input-specific churn. Table 10 reports detection on the three character-perturbation slices, which concentrate near the decision boundary.

Table 5: Native integer execution versus simulated INT8. Flip rates are measured against each path’s own FP32 reference (PyTorch FP32 for INT8 PyTorch; ONNX Runtime FP32 for INT8 ONNX RT). J : Jaccard overlap of the two flip sets; Agree: fraction of identical final decisions between the native and simulated models.

Detector	Native	Simulated	Flip _{nat} %	Flip _{sim} %	J	Agree %
ProtectAI-v2	INT8 PyTorch	W8A8 ^s (token)	30.05	1.78	0.02	69.69
	INT8 PyTorch	W8A8 ^s (tensor)	30.05	30.60	0.82	94.01
	INT8 ONNX RT	W8A8 ^s (token)	38.19	1.78	0.01	60.76
	INT8 ONNX RT	W8A8 ^s (tensor)	38.19	30.60	0.67	86.24
	INT8 PyTorch (per-ch.)	W8A8 ^s (token)	13.97	1.78	0.05	85.77
	INT8 PyTorch (per-ch.)	W8A8 ^s (tensor)	13.97	30.60	0.26	73.79
	INT8 ONNX RT (per-ch.)	W8A8 ^s (token)	13.50	1.78	0.04	85.88
	INT8 ONNX RT (per-ch.)	W8A8 ^s (tensor)	13.50	30.60	0.27	74.41
PIGuard	INT8 PyTorch	W8A8 ^s (token)	31.54	3.05	0.01	65.92
	INT8 PyTorch	W8A8 ^s (tensor)	31.54	31.54	1.00	100.00
	INT8 ONNX RT	W8A8 ^s (token)	31.54	3.05	0.01	65.92
	INT8 ONNX RT	W8A8 ^s (tensor)	31.54	31.54	1.00	100.00
Deepset	INT8 PyTorch	W8A8 ^s (token)	10.13	2.00	0.20	91.87
	INT8 PyTorch	W8A8 ^s (tensor)	10.13	10.45	0.90	98.87
	INT8 ONNX RT	W8A8 ^s (token)	10.34	2.00	0.19	91.65
	INT8 ONNX RT	W8A8 ^s (tensor)	10.34	10.45	0.94	99.38
FMOps	INT8 PyTorch	W8A8 ^s (token)	1.23	0.36	0.19	98.91
	INT8 PyTorch	W8A8 ^s (tensor)	1.23	1.85	0.23	98.08
	INT8 ONNX RT	W8A8 ^s (token)	0.54	0.36	0.04	99.17
	INT8 ONNX RT	W8A8 ^s (tensor)	0.54	1.85	0.08	97.97

Table 6: Granularity ablation for ProtectAI-v2 on a 150-input subset: flip rate against FP32 and mean absolute change in attack probability (simulated quantization).

Weights / activations	Flip %	mean $ \Delta p $
Per-channel / per-token (W8A8 ^s token)	1.3	0.017
Per-channel / per-tensor 7-bit	13.3	0.161
Per-tensor / per-token	35.3	0.370
Per-tensor / per-tensor 7-bit (native-like)	28.0	0.349
Per-tensor / per-tensor 8-bit	36.7	0.399
Per-tensor / none (weights only)	36.7	0.376
None / per-tensor 7-bit (activations only)	21.3	0.212
Native-like, FFN-output layers kept FP32	14.0	0.119
Native-like, classifier head kept FP32	28.0	0.349

5.6. Compute and runtime environment

The first native, RTN and simulated runs were executed on a 2-vCPU aarch64 CPU (PyTorch 2.14.1+cpu, Transformers 5.19.0, ONNX Runtime 1.30.0). The GPTQ and AWQ-style runs, the PIGuard grid, Deepset and the remaining configurations, together with the x86_64 replicates, ran as a 26-job matrix on GitHub Actions x86_64 CPU runners (AMD EPYC, 2 vCPUs, same library versions). No GPU was used. The platform and software versions of each run are recorded in its metadata file. FP32 of ProtectAI-v2 and FMOps was run on both platforms with identical decisions.

6. Discussion

6.1. Why aggregate checks miss churn

Aggregate metrics are sums over inputs, and compression errors are not uniformly signed: a perturbation of the logits pushes some near-boundary inputs across τ in one direction and others in the opposite direction. When the two directions are roughly balanced, accuracy and F1 are preserved while the identity of the misclassified inputs changes—the McNemar discordant pairs b and c in Table 4 are both large while their difference is not significant. AUROC is even less sensitive, because it depends only on the ranking of scores and is unaffected by uniform shifts. A guardrail operator, however, experiences churn as concrete events: an attack that was previously blocked now reaches the LLM, or a user request that previously succeeded is now refused. Neither is visible in a benchmark score.

Table 7: Native PyTorch dynamic INT8 on the aarch64 CPU and on x86_64 CI runners (flip rate against each platform’s FP32; identical FP32 decisions on both).

Detector	Flip aarch64 %	Flip x86_64 %	Missed (a/x)	Differing decisions
ProtectAI-v2	30.05	29.73	514/507	117
FMOps	1.23	1.34	7/7	7

Table 8: Calibration-based 4-bit quantization versus RTN (W4 per-channel unless noted). GPTQ/AWQ values are mean $\pm$ sample std over calibration seeds (128 calibration prompts per seed drawn from training splits disjoint from the evaluation set). J_{seed} : mean pairwise Jaccard overlap of flip sets across seeds; J_{RTN} : mean overlap with the RTN flip set.

Detector	Method	Seeds	Flip %	Missed	New FP	F1	J_{seed}	J_{RTN}
ProtectAI-v2	RTN	–	3.92	51	10	0.895	–	–
	AWQ	1	3.01	18.0	10.0	0.907	–	0.35
	GPTQ (g128)	1	1.31	10.0	7.0	0.888	–	0.32
	GPTQ	3	9.39 $\pm$ 0.67	184.0 $\pm$ 14.80	1.0 $\pm$ 1.73	0.785	0.66	0.24
PIGuard	RTN	–	5.74	66	24	0.787	–	–
	GPTQ	3	15.69 $\pm$ 0.95	385.3 $\pm$ 23.76	0.0 $\pm$ 0.00	0.686	0.76	0.16
Deepset	RTN	–	2.00	6	22	0.686	–	–
	GPTQ	3	3.91 $\pm$ 1.29	43.7 $\pm$ 40.05	57.0 $\pm$ 7.00	0.680	0.29	0.15
FMOps	RTN	–	0.80	0	11	0.673	–	–
	AWQ	1	0.69	2.0	11.0	0.672	–	0.46
	GPTQ	3	0.64 $\pm$ 0.23	0.7 $\pm$ 1.15	12.3 $\pm$ 4.93	0.673	0.31	0.35

6.2. Simulation is not a substitute for the deployed artifact

The native INT8 results are the most practically important finding of this study. The default one-line dynamic INT8 paths of PyTorch and ONNX Runtime quantize each weight matrix with a single per-tensor scale, and the transformer layers of the DeBERTa-v3 detectors contain weight and activation outliers for which a single scale is too coarse. Simulated per-channel/per-token INT8—the variant most papers and blog posts use to argue that “INT8 is lossless”—changed only a few percent of decisions, whereas the native artifacts that an operator would actually deploy changed far more. Our granularity ablation (Section 5.2) shows that per-tensor weights alone (36.7% flips) and per-tensor activations alone (21.3%) are each sufficient to cause the collapse; the two effects do not simply add up, since combining them gave 28.0–36.7%. The lesson is general: a simulation is only as good as its match to the kernel that will run in production, and the acceptance test must be executed on the exported artifact itself.

6.3. Calibration-based quantization

GPTQ and AWQ minimise layer-wise reconstruction error, but in our experiments (Section 5.4) this did not translate into fewer flipped decisions: GPTQ flipped more decisions than RTN for three of four detectors, and the flipped inputs differed across calibration seeds. The practical implication of seed-dependent flip sets is that two “identical” quantized releases, produced with the same recipe but different calibration samples, are not interchangeable from a security standpoint: red-team results obtained on one do not transfer to the other.

6.4. Recommendations

1. **Validate agreement, not just accuracy.** Report the flip rate relative to the full-precision parent with a confidence interval, and separately the number of newly missed attacks and newly blocked benign inputs. Treat any newly missed attack in a curated attack suite as a release blocker.
2. **Test the artifact you ship.** Run the acceptance suite on the exported, quantized artifact with the production runtime and kernel library, not on a fake-quantized proxy. Per-tensor dynamic INT8 should not be assumed safe for DeBERTa-v3 classifiers.
3. **Include adversarial and over-defence slices.** Character-perturbed attacks and trigger-word benign prompts (e.g., NotInject) concentrate near the decision boundary, where compression-induced drift is largest.
4. **Re-calibrate per artifact, but do not rely on it.** Re-tuning τ on held-out data removes global score shifts, but not input-specific churn; thresholds tuned for the FP32 model must never be reused blindly.
5. **Prefer finer granularity.** Per-channel or per-group weight scales and per-token activation scales were consistently safer than per-tensor scales; BF16 was effectively lossless.
6. **Treat each compressed artifact as a new model** for red-teaming, incident response, and audit purposes, and pin the exact quantization recipe and calibration sample in the deployment record.

Table 9: Held-out flip rate (%) at the default threshold and after re-tuning τ to maximize agreement with FP32 on a disjoint half; mean $\pm$ std over 5 random 50/50 splits. τ^* : range of selected thresholds.

Detector	Format	$\tau=0.5$	τ^*	τ^* range
ProtectAI-v2	W8A8 ^s (token)	1.87 $\pm$ 0.13	1.17 $\pm$ 0.14	0.80–0.94
	INT8 PyTorch	29.68 $\pm$ 1.00	29.12 $\pm$ 1.03	0.27–0.42
	INT8 ONNX RT	37.91 $\pm$ 1.41	30.68 $\pm$ 0.73	0.17–0.17
	W4-g128 RTN	1.52 $\pm$ 0.19	1.12 $\pm$ 0.10	0.74–0.78
	W4-pc RTN	3.69 $\pm$ 0.23	3.65 $\pm$ 0.65	0.01–0.10
	W4-pc GPTQ (s0)	9.81 $\pm$ 0.36	4.92 $\pm$ 0.62	0.01–0.01
PIGuard	W8A8 ^s (token)	3.15 $\pm$ 0.19	2.22 $\pm$ 0.27	0.53–0.58
	INT8 PyTorch	30.71 $\pm$ 0.83	30.41 $\pm$ 0.83	0.24–0.27
	INT8 ONNX RT	30.71 $\pm$ 0.83	28.92 $\pm$ 0.69	0.17–0.19
	W4-g128 RTN	2.18 $\pm$ 0.23	2.35 $\pm$ 0.15	0.46–0.51
	W4-pc RTN	5.58 $\pm$ 0.41	5.66 $\pm$ 0.30	0.52–0.61
	W4-pc GPTQ (s0)	15.80 $\pm$ 0.61	10.99 $\pm$ 0.80	0.20–0.23
Deepset	W8A8 ^s (token)	1.90 $\pm$ 0.19	0.96 $\pm$ 0.26	0.95–0.98
	INT8 PyTorch	10.40 $\pm$ 0.82	10.45 $\pm$ 0.77	0.46–0.53
	INT8 ONNX RT	10.60 $\pm$ 0.67	8.67 $\pm$ 0.70	0.65–0.65
	W4-g128 RTN	1.37 $\pm$ 0.13	0.50 $\pm$ 0.07	0.95–0.97
	W4-pc RTN	2.01 $\pm$ 0.34	1.96 $\pm$ 0.31	0.28–0.56
	W4-pc GPTQ (s0)	5.40 $\pm$ 0.37	3.92 $\pm$ 0.45	0.29–0.35
FMOps	W8A8 ^s (token)	0.39 $\pm$ 0.16	0.28 $\pm$ 0.10	0.32–0.40
	INT8 PyTorch	1.54 $\pm$ 0.29	1.49 $\pm$ 0.26	0.10–0.32
	INT8 ONNX RT	0.58 $\pm$ 0.18	0.58 $\pm$ 0.19	0.27–0.74
	W4-g128 RTN	0.87 $\pm$ 0.22	0.82 $\pm$ 0.29	0.53–0.92
	W4-pc RTN	0.83 $\pm$ 0.20	0.96 $\pm$ 0.17	0.36–0.71
	W4-pc GPTQ (s0)	0.64 $\pm$ 0.20	0.41 $\pm$ 0.04	0.59–0.71

6.5. Implications for guardrail vendors and evaluators

Vendors that publish quantized or ONNX variants of a detector should publish their agreement with the reference model, not only benchmark accuracy, and benchmark maintainers could add an “agreement with reference” track for compressed variants. For auditors, a compressed guardrail with unchanged F1 but a material flip rate is a different control and should be re-assessed.

7. Threats to validity

Internal validity. Every comparison is paired: each compressed model is compared with the same detector’s FP32 decisions on identical inputs, tokenisation, truncation, and batching, so differences cannot be explained by data or pre-processing. Batch composition can affect floating-point results through padding; we use identical length-sorted batches for all simulated configurations and batch size 1 for native configurations, and we report the FP32 references of each execution path. Re-running FP32 in a different software environment changed no decision. The simulated formats (W8A8^s, W4 RTN, GPTQ, AWQ-style) are quantize–dequantize emulations: they reproduce rounding of weights and activations but not integer accumulation, requantisation of intermediate results, or operator fusion. This is exactly why we also execute two native INT8 paths and compare their flip sets with the simulations. Our GPTQ and AWQ-style implementations are re-implementations for encoder classifiers rather than the reference LLM toolkits; we log the GPTQ-to-RTN ratio of the layer-wise proxy loss to confirm that GPTQ reduces reconstruction error as intended.

Construct validity. The flip rate treats every changed decision equally, whereas operators weigh a missed attack and a false alarm differently; we therefore report the two directions separately. Labels in public prompt-injection datasets are noisy, and some “benign” Safe-Guard or Jailbreak-Classification prompts are arguably attacks; this affects accuracy-type metrics but not flip rates, which are label-free. Detection rate on character-perturbed slices measures robustness to non-adaptive perturbations only.

External validity. We study four detectors, three of which share the DeBERTa-v3-base architecture; results may differ for larger guardrails (e.g., LLM-based classifiers [16]), for detectors trained with quantization-aware training, or on other hardware and kernel libraries. Native INT8 was measured on an aarch64 CPU and, for ProtectAI-v2 and FMOps, replicated on x86_64 CPU runners (same flip rate within 0.4 points, but 4.25% and 0.25% of decisions differed); `fbgemm` kernels or GPU INT8 kernels were not tested and may produce different flip sets. Experiments ran on a local 2-vCPU aarch64 CPU and on GitHub Actions x86_64 CPU runners. The evaluation set mixes public benchmarks, some of which may overlap the detectors’ training data, which

Table 10: Detection rate (%) on character-perturbed attack slices (300 attacks each).

Detector	Format	Homoglyph	Zero-width	Leet
ProtectAI-v2	FP32	96.0	100.0	91.7
	BF16	96.3	100.0	91.3
	W8A8 ^s (token)	98.7	100.0	95.7
	INT8 PyTorch	68.3	95.7	94.0
	INT8 ONNX RT	23.3	99.7	42.0
	W4-g128 RTN	95.0	100.0	93.7
	W4-pc RTN	88.7	100.0	88.7
	W4-pc GPTQ (s0)	80.0	100.0	83.3
PIGuard	FP32	18.0	0.3	84.0
	BF16	18.0	0.3	85.0
	W8A8 ^s (token)	24.3	1.0	91.0
	INT8 PyTorch	0.0	0.0	0.0
	INT8 ONNX RT	0.0	0.0	0.0
	W4-g128 RTN	16.3	0.0	87.3
	W4-pc RTN	18.3	0.0	85.7
	W4-pc GPTQ (s0)	0.7	0.0	5.3
Deepset	FP32	99.7	100.0	100.0
	W8A8 ^s (token)	100.0	100.0	100.0
	INT8 PyTorch	100.0	100.0	100.0
	INT8 ONNX RT	100.0	100.0	100.0
	W4-g128 RTN	99.7	100.0	100.0
	W4-pc RTN	99.7	100.0	100.0
	W4-pc GPTQ (s0)	71.7	99.7	100.0
FMOps	FP32	100.0	98.7	100.0
	W8A8 ^s (token)	100.0	98.3	100.0
	INT8 PyTorch	100.0	97.7	100.0
	INT8 ONNX RT	100.0	98.7	100.0
	W4-g128 RTN	100.0	99.3	100.0
	W4-pc RTN	100.0	98.7	100.0
	W4-pc GPTQ (s0)	100.0	99.0	100.0

inflates absolute accuracy for some detectors; again this does not affect the paired change relative to FP32. The perturbation slices use one random seed each.

Conclusion validity. We report 95% bootstrap intervals for flip rates and F1, exact McNemar tests for paired correctness, and mean $\pm$ standard deviation over three calibration seeds and five re-calibration splits. Three seeds give only a coarse estimate of seed variance. We do not correct McNemar p -values for multiple comparisons; our conclusions rest on effect sizes (numbers of flipped decisions) rather than on any single test.

8. Conclusion

Accuracy-neutral compression is not decision-neutral. On four public prompt-injection detectors, unchanged aggregate F1 coexisted with flip rates of up to 15.69% (GPTQ, PIGuard) and, for the default native per-tensor INT8 paths, collapse of the classifier (ProtectAI-v2 AUROC 0.977 to 0.542 and 0.505; PIGuard F1 0.807 to 0.000). A per-token simulation understated that risk by an order of magnitude (1.78% and 3.05% flips), while a per-tensor simulation matched it; the ablation shows that per-tensor weight scales alone (36.7%) and per-tensor activation scales alone (21.3%) are each sufficient to cause it. Native results transferred across an aarch64 and an x86_64 CPU in aggregate but not input by input. Calibration-based 4-bit methods were not a reliable remedy: GPTQ increased flips relative to RTN for ProtectAI-v2, PIGuard and Deepset, and its flip sets varied across calibration seeds. We therefore recommend validating compressed guardrails by paired decision agreement with the parent model on the exported artifact, reporting newly missed attacks explicitly, preferring per-channel or per-group scales, and treating each compressed artifact as a new model. Limitations include four detectors, a single decision threshold, three calibration seeds, and CPU-only measurements; extending the study to LLM-based guardrails, quantization-aware training and GPU kernels is future work.

Data and code availability

All detectors and datasets used are public. Code and per-input scores for an earlier version of this study are available at <https://github.com/horizonbymuneeb/quantized-guardians> and archived on Zenodo

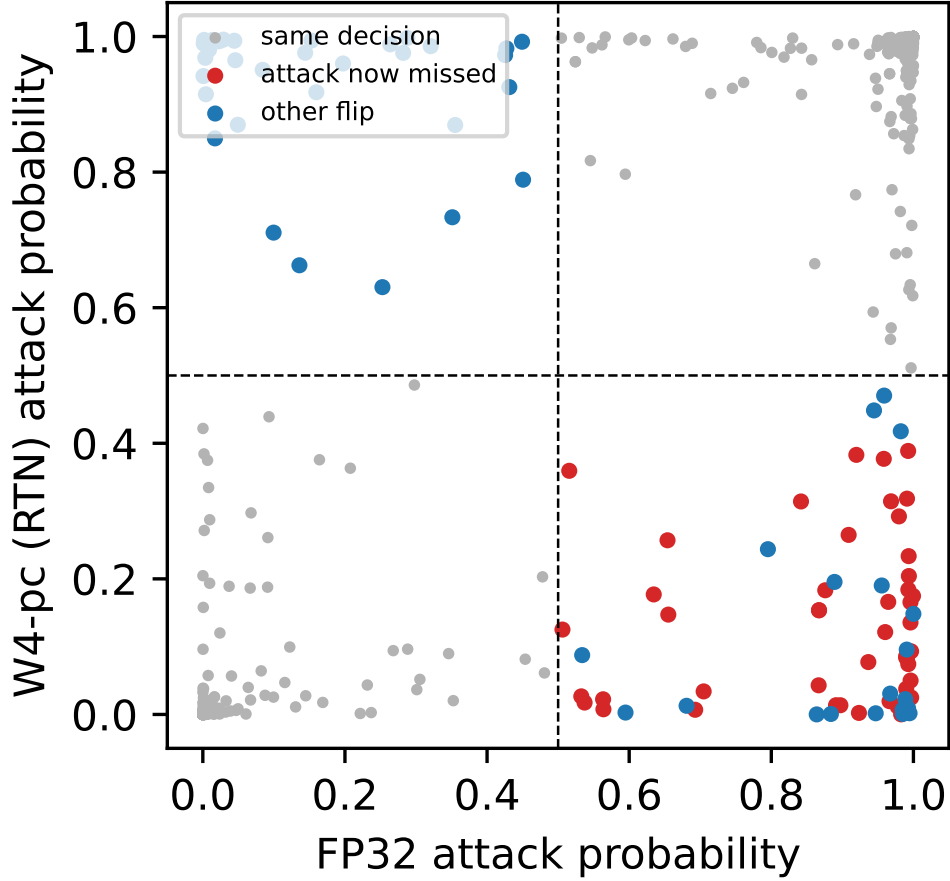


Figure 2: Aggregate F1 change versus decision flip rate across configurations.

(<https://doi.org/10.5281/zenodo.23210534>). The complete code, run logs, and every per-input score underlying this manuscript will be released in the same repository and archive upon publication, and are available to the editor and reviewers on request.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used AI tools to assist with running experiments and drafting the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

CRedit authorship contribution statement

Muneeb Anjum: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] F. Perez, I. Ribeiro, Ignore previous prompt: Attack techniques for language models, in: NeurIPS ML Safety Workshop, 2022.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.
- [3] Y. Liu, Y. Jia, R. Geng, J. Jia, N. Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses, in: Proc. 33rd USENIX Security Symposium, 2024.
- [4] P. He, J. Gao, W. Chen, DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing, in: Proc. ICLR, 2023.
- [5] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, arXiv preprint arXiv:1910.01108 (2019).
- [6] ProtectAI.com, Fine-tuned DeBERTa-v3-base for prompt injection detection (v2), Hugging Face model card, <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2> (2024).
- [7] H. Li, X. Liu, PIGuard: Prompt injection guardrail via mitigating overdefense for free, in: Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL), 2025, earlier version: InjecGuard, arXiv:2410.22770.
- [8] A. Dubey, et al., The Llama 3 herd of models, arXiv preprint arXiv:2407.21783 (2024).
- [9] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit matrix multiplication for transformers at scale, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [10] E. Frantar, S. Ashkboos, T. Hoeffer, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: Proc. ICLR, 2023.
- [11] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proc. Machine Learning and Systems (MLSys), 2024.
- [12] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han, SmoothQuant: Accurate and efficient post-training quantization for large language models, in: Proc. ICML, 2023.
- [13] M. M. Fard, Q. Cormier, K. Canini, M. Gupta, Launch and iterate: Reducing prediction churn, in: Advances in Neural Information Processing Systems (NeurIPS), 2016.
- [14] A. Dutta, S. Krishnan, N. Kwatra, R. Ramjee, Accuracy is not all you need, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [15] S. Hooker, A. Courville, G. Clark, Y. Dauphin, A. Frome, What do compressed deep neural networks forget?, arXiv preprint arXiv:1911.05248 (2019).
- [16] H. Inan, et al., Llama Guard: LLM-based input-output safeguard for human-AI conversations, arXiv preprint arXiv:2312.06674 (2023).
- [17] W. Hackett, L. Birch, S. Trawicki, N. Suri, P. Garraghan, Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems, in: Proc. 1st Workshop on LLM Security (LLMSEC), 2025, arXiv:2504.11168.
- [18] M. J. Rahman, I. Alouani, Bypassing prompt injection detectors through evasive injections, arXiv preprint arXiv:2602.00750 (2026).
- [19] Y. Zhou, C. Zhu, J. Wang, X. He, Y. Zhai, K. Sun, M. Wei, J. Xiong, Prompt overflow: What the guardrail inspects is not what the model infers, arXiv preprint arXiv:2605.23196 (2026).
- [20] B. Jacob, et al., Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: Proc. CVPR, 2018.
- [21] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, T. Blankevoort, A white paper on neural network quantization, arXiv preprint arXiv:2106.08295 (2021).

- [22] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, K. Keutzer, A survey of quantization methods for efficient neural network inference, arXiv preprint arXiv:2103.13630 (2021).
- [23] O. Zafrir, G. Boudoukh, P. Izsak, M. Wasserblat, Q8BERT: Quantized 8bit BERT, in: 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC2) at NeurIPS, 2019.
- [24] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, K. Keutzer, Q-BERT: Hessian based ultra low precision quantization of BERT, in: Proc. AAAI, 2020.
- [25] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, K. Keutzer, I-BERT: Integer-only BERT quantization, in: Proc. ICML, 2021.
- [26] A. Paszke, et al., PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [27] ONNX Runtime developers, ONNX Runtime, <https://onnxruntime.ai> (2021).
- [28] S. Hooker, N. Moorosi, G. Clark, S. Bengio, E. Denton, Characterising bias in compressed models, arXiv preprint arXiv:2010.03058 (2020).
- [29] J. Hong, et al., Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression, in: Proc. ICML, 2024.
- [30] K. Egashira, M. Vero, R. Staab, J. He, M. Vechev, Exploiting LLM quantization, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [31] J. Lin, C. Gan, S. Han, Defensive quantization: When efficiency meets robustness, in: Proc. ICLR, 2019.
- [32] deepset, prompt-injections (dataset), Hugging Face, <https://huggingface.co/datasets/deepset/prompt-injections> (2023).
- [33] xTRam1, safe-guard-prompt-injection (dataset), Hugging Face, <https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection> (2024).
- [34] jackhhao, jailbreak-classification (dataset), Hugging Face, <https://huggingface.co/datasets/jackhhao/jailbreak-classification> (2023).
- [35] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proc. ICML, 2017.
- [36] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
- [37] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.